Accelerating the Training of Tree-Based Machine Learning Algorithms for Edge Systems

Accelerating the Training of Tree-Based Machine Learning Algorithms for Edge Systems

Ali Emre Oztas*, Mahir Demir*, Can Erim Karsal†, Nikolaos Kyparissas*, and Mikel Luján*
*University of Manchester, UK
†Technical University of Munich, Germany
{aliemre.oztas, mahir.demir, nikolaos.kyparissas, mikel.lujan}@manchester.ac.uk, can.karsal@tum.de

***Abstract*—Tree-based algorithms, such as XGBOOST – an open source implementation of Gradient Boosted Decision Trees (GBDT) – are ubiquitous for many Machine Learning (ML) tasks, especially when dealing with tabular/structured data and requiring explainability. As dataset size continues to grow, training ensembles of trees (random forest, XGBOOST, CatBoost, LightGBM) has been parallelized on multi-core systems, accelerated on GPUs, and distributed on computer clusters. However, FPGA acceleration targeting the training of tree-based ML remains underexplored.**

Analyzing the tree structure of trained GBDT models, we observe a significant overlap of the features used as nodes at the top levels of the ensemble. Guided by this insight, we modify the training algorithm of GBDT to harness the computational redundancy associated with the top nodes, as well as to create a novel accelerator architecture, FaGBM. For training, FaGBM reduces end-to-end GBDT training latency by up to 2× speedup over NVIDIA Jetson Thor with XGBOOST, LightGBM, and CatBoost. Moreover, for GBDT, FaGBM employs a novel adaptive approximate division to compute the split gain, which reduces LUT usage by up to 40K. For Decision Trees training, this paper investigates bit-wise approximate logarithms, resulting in an 87% reduction in DSP usage in FPGAs compared to a fixed-point implementation. The experiments further demonstrate that FaGBM preserves model accuracy while achieving significant energy efficiency over optimized multi-core and GPU baselines.

***Index Terms*—Decision tree, Gradient boosting, FPGA, hardware acceleration**

I. Introduction

The enduring popularity of tree-based machine learning algorithms began with ID3 [1] and CART [2]. Initially, these algorithms were used to train a single Decision Tree (DT) for classification or regression. Fast-forward, and boosted ensembles of trees are ubiquitous in supervised Machine Learning (ML) applications [3], [4], especially when dealing with tabular/structured data and requiring explainability.

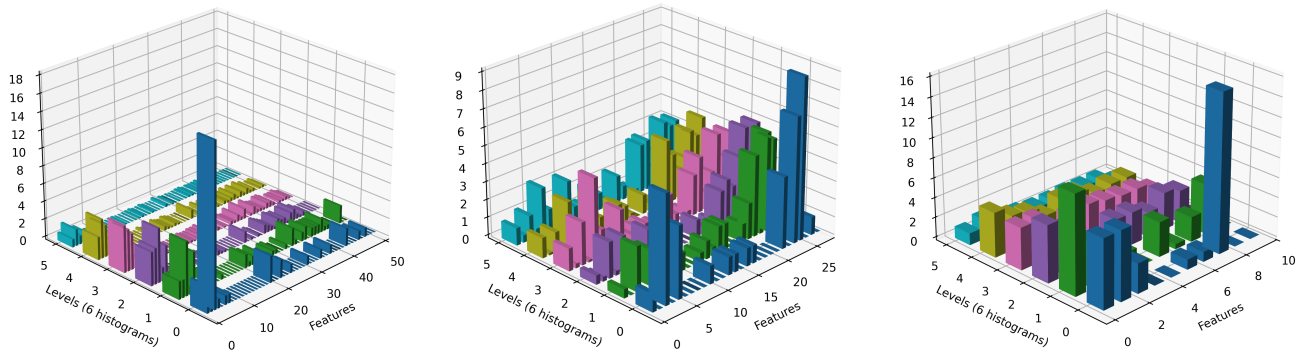
Ensemble models combine weak learners — each only slightly better than random guessing — into a strong model. In *bagging*, multiple trees are trained on different subsets of the data and combined by averaging or majority voting; the best-known example is *Random Forest* (RF) [5]. *Boosting* [6], [7] instead trains each learner to correct the mistakes of the others. Its most popular form, Gradient Boosted Decision Trees (GBDT), is often referred to by its open-source implementation, XGBoost [8], which has become indispensable for structured data across domains from recommendation systems to medical analysis [9], [10].

As the dataset continues to grow in size (rows/samples) and dimensionality (number of features), training ensembles of trees has been parallelized on multi-core systems, accelerated on GPUs, and distributed across clusters (e.g. Yandex's CatBoost [11] and Microsoft's LightGBM [12]). From a hardware design point of view, the research literature has studied mainly the acceleration of inference of tree-based models with ASICs and FPGAs [13], [14], [15], [16], [17], [18], [19]. For example, some FPGA research frameworks can convert the tree models generated by GBDT into RTL to accelerate the inference on FPGAs [15], [20]. Nonetheless, FPGA acceleration of training these ensembles of trees (e.g. GBDT) is underexplored [21], [22], [23], especially when considering edge systems.

Training ensembles of trees starts with building a tree. Oversimplifying, each tree is constructed recursively by selecting one feature and calculating a *split criterion*. The split criterion aims to maximize the accuracy. Effectively, each node uses the feature and the condition to split the training dataset. The top node (*root*) represents the feature that provides the largest information gain (reduction in uncertainty) and can also be interpreted as the strongest correlated feature. A *leaf* node represents the output (e.g. classification, regression, or ranking) of the generated tree-based ML model.

To train an individual tree, there are different optimization opportunities depending on the specific algorithm. For DTs and RF, *mutual information gain* is widely used as the split criterion. Such a criterion computes the Entropy [24] and involves logarithmic operations, which can be costly in terms of hardware utilization when requiring full accuracy. On the other hand, there is a known family of bit-wise approximate logarithms [25] that relies upon bit-level operations instead of standard floating-point, which is more hardware resource friendly. The impact of bit-wise logarithms on the accuracy of the ML model and hardware cost trade-off has not been studied yet.

For tree-based ensembles, the split criterion for each feature (node) could be computed repeatedly across the training of weak learner trees. For example, in GBDT, this paper demonstrates empirically that levels close to the root nodes tend to repeat a subset of features more often than not. Such structural overlap can be considered intuitive, given that the top nodes should carry the most information gain for the specific ML task at hand. Figure 1 illustrates the *node overlap* across the ensembles for three different datasets using XGBoost with



(a) Bank (b) Higgs (c) Magic

Fig. 1: Analysis of trees created by XGBOOST – z-axis denotes how many times each feature appears at a given tree level across the ensemble, normalized by node count of that level.

40 trees and a tree depth of 6; parameters set according to [21], [22]. For example, in the Bank dataset, Figure 1 (a), the feature labeled 2 appears as the top node in 18 of the 40 trees, and feature 1 recurs at multiple levels across the ensemble. As introduced in Section II, computing the information gain involves three mathematical divisions. However, there is no analysis in the literature of the trade-off between the hardware cost of approximating the divisions and the overall accuracy for tree-based ML algorithms.

Given the above context, this paper presents FaGBM (FPGA-accelerated Gradient Boosting Machine) and makes the following three novel contributions:

- We design and implement FaGBM, including a novel algorithm that takes advantage of the node overlap in the tree ensemble, and evaluate the effectiveness against LightGBM, CatBoost, and XGBOOST for edge systems.
- We evaluate for the first time hardware-efficient logarithmic approximations for calculating entropy (part of the information gain) in training algorithms for DT and RF.
- To alleviate the hardware utilization burden of the division operation part of GBDT training, we propose and evaluate an adaptive selection of an approximate division depending on the magnitude of the denominator.

The remainder of the paper is organized as follows. Section II describes the training of DTs, RF, and GBDT, along with relevant related work. Section III presents the design and implementation of FaGBM, which specializes per dataset — a good match for edge systems, where data is local (privacy-conscious, no cloud offload), the ML tasks are known ahead of time, and datasets grow or update at constant size. Section IV evaluates FaGBM on AMD Versal and Kintex FPGAs against optimized multi-core and NVIDIA Jetson Thor baselines. Section V concludes this paper.

II. BACKGROUND

When dealing with continuous features, it is standard practice to discretize the features into a set of *bins*. Thus, it is possible to compress an 8-byte floating-point number into 256 bins, which only requires 1 byte of storage. For example, Figure 1 shows the results of an XGBOOST model trained on features represented with 256 bins. Furthermore, using bins reduces the number of possible splits, making it more tractable.

Normally, the training algorithms make a full pass to count the occurrences for each feature and its corresponding bin, *(feature, bin) pair*, against the target label (or labels) that the resulting ML model is trying to predict. Counting is used as an approximation to capture the probability of a given label given a particular feature with a value as represented in the bin. Thus, histograms are a key data structure to efficiently represent the outcome of counting (i.e. estimate the probability). For DTs and RF, histograms are used in the calculation of the information gain; see Section II-A. For GBDT, histograms are also used to store the gradient and Hessian accumulations efficiently; see Section II-B.

A. Training – Decision Trees and Random Forest

Although the training algorithm for a Decision Tree (DT) can either be recursive [1] or iterative [2], the latter is most suitable for hardware design. The iterative approach trains the tree one level at a time (*level-wise*) and starts with an empty tree. It iteratively applies the same training steps to each level. The training comprises two phases, as shown in Algorithm 1. The first phase (Lines 3-11), for each sample in the dataset, navigates the current tree and finds the current leaf node to which it belongs. Then, histograms for that leaf node are updated based on the pair of feature and bin values.

The second phase (Lines 12-26) calculates the information gain for each feature-bin split at each leaf node. The information gain is calculated as the difference between the entropy of the leaf node and the sum of the entropy of its possible child nodes. The entropy [24] of a node is given by Equation 1, where N_0, N_1 stand for the Label 0 and Label 1 instance counts of the node as $N = N_0 + N_1$.

$$H(child) = -\left(\frac{N_0}{N}\log\frac{N_0}{N} + \frac{N_1}{N}\log\frac{N_1}{N}\right). \quad (1)$$

The best split with the highest gain is selected, and based on that split, two new child leaves are added to the tree.

Algorithm 1: Iterative Level-Wise Decision Tree Training

```
Data: Dataset D with features A, bins per feature B, max
      depth d_max
Result: Decision Tree T
1  Initialize root node;
2  ActiveLeaves ← {root};
3  for depth ← 0 to d_max − 1 do
4      Initialize histograms H[leaf][feature][bin] ← 0;
5      foreach instance x ∈ D do
6          Find leaf L in ActiveLeaves where x currently falls;
7          foreach feature a ∈ A do
8              Determine bin b of x[a];
9              Increment H[L][a][b];
10         end
11     end
12     NewLeaves ← ∅;
13     foreach leaf L ∈ ActiveLeaves do
14         (*a, *b, *bestGain) ← (None, None, −∞);
15         foreach feature a ∈ A do
16             for b ← 1 to B do
17                 gain = InformationGain(H[L][a][b]);
18                 if gain > bestGain then
19                     (*a, *b, *g) ← (a, b, gain);
20                 end
21             end
22         end
23         Split leaf L using (*a, *b);
24         Add produced children to NewLeaves;
25     end
26     ActiveLeaves ← NewLeaves;
27 end
28 return Tree T;
```

Among these two phases, the first one is memory-bound due to the low arithmetic intensity, and the second one is relatively compute-bound due to the series of entropy calculations within nested loops.

The related work proposes different approaches to mitigate these bottlenecks. Xilinx Data Analytics Library includes a DT Accelerator, which processes multiple leaf nodes in parallel [26]. Histograms are stored in UltraRAM (URAM). However, URAMs have limited read/write ports, which forces histogram updates to be serialized across samples. Thus, parallelism takes place across nodes and candidate splits. The Data Analytics design implements only 2-way parallelism for node split calculations. This combination of factors limits the throughput and constrains the supported feature and split counts (64 features and 128 splits in [26]).

Histogram storage is a critical FPGA-specific challenge. Normally, URAMs and BRAMs are preferred due to their capacity. However, when histogram updates are performed in a map-reduce fashion, multiple samples are processed in parallel, and their histogram updates are accumulated independently. When this operation is performed in parallel for both features and bins, it is not feasible using URAM and BRAM storage. In such cases, the synthesizer maps histograms to distributed RAM (LUTRAM), which supports multiple parallel writes at the cost of high LUT utilization, limiting scalability. Lin *et al.* [27] addressed the same memory issue by changing histograms to quantile-based summaries. Using this technique, they reduce memory usage to a constant and avoid complex arithmetic.

Moreover, the split evaluation phase (Lines 12-26) introduces an additional hardware complexity. In this phase, computing information gain can be parallelized across bins or features to increase throughput, but this significantly increases area due to the logarithmic operations. Implementing fixed-point logarithms at high parallelism significantly increases DSP and LUT usage, limiting scalability. To alleviate this, FaGBM considers bit-wise logarithm approximations, which substantially reduce area while preserving sufficient accuracy for split selection (see Section III). RF incurs exactly the same challenges as the DT algorithm, albeit RF trains multiple trees using bagging.

B. Training – Gradient Boosted Decision Trees

While training Gradient Boosted Decision Trees (GBDT), the ensemble is constructed incrementally, one DT at a time. Each new tree is trained to correct the errors made by the current ensemble of trees. This is achieved by computing the first- and second-order derivatives of the loss function for each sample, namely the *gradient* g_i and the *Hessian* h_i. Whereas RF promotes ensemble diversity by training each tree on a different part of the training dataset and randomly selecting the subset of features allowed at each tree level, GBDT enables the split criteria to be based on the overall loss reduction of the growing ensemble. Algorithm 2 presents the steps underlying GBDT.

$$\mathcal{L} = \frac{1}{2}\left(\frac{\left(\sum_{i\in I_L} g_i\right)^2}{\left(\sum_{i\in I_L} h_i\right)+\lambda} + \frac{\left(\sum_{i\in I_R} g_i\right)^2}{\left(\sum_{i\in I_R} h_i\right)+\lambda} - \frac{\left(\sum_{i\in I} g_i\right)^2}{\left(\sum_{i\in I} h_i\right)+\lambda}\right). \quad (2)$$

Using the best splits, the current tree is constructed, and predictions are updated by inferring on the entire dataset (encapsulated under the `BuildTree` function).

GBDT shares the already described challenges regarding histogram memory and area consumption. A unique challenge in GBDT is the order of the steps. From a hardware perspective, after calculating gradients and Hessians, the sample feature values are no longer available. Thus, the prediction update phase requires additional looping over each sample, which doubles the runtime. Hardware designers must manipulate data access patterns to alleviate this inefficiency.

ThunderGBM (academic) [21], [28] and LightGBM (Microsoft) [12] introduced a set of techniques to execute GBDT training efficiently on GPUs, and these have influenced the improvements of XGBOOST maintained by NVIDIA, and CatBoost maintained by Yandex [11]. These three are the most efficient implementations of GDBT for GPUs. He *et al.* [22] accelerated GBDT training, designing an ASIC, known as Booster, and parallelized the histogram computations using 3200 small SRAM-based processing units – simple compute logic (adders, counters) – working in parallel and arranged

in a grid. However, their approach incurs significant energy and area overheads. Tanaka *et al*. [23] is, to the best of our knowledge, the only available[1] FPGA research for training GBDT. Tanaka *et al*. defined an engine as a specialized FPGA hardware unit that implements a specific stage of GBDT within a pipelined architecture. Tanaka *et al*. implemented engines for the data scan, split evaluation, best split selection, tree update, and partitioning steps of GBDT. The design was limited to two pipelines synthesized side by side. Also, the size of the datasets it can handle is limited by the URAM size of the FPGA, which corresponds to 1.08M samples for the HIGGS dataset when that dataset contains 11M. To sum up, none of the previous work (neither GPU nor FPGA/ASIC) has explored exploiting the structural similarity of trees (faster execution), nor how to approximate the division computation (reduce resources) to optimize GBDT training.

Algorithm 2: Gradient Boosted Decision Trees Training

Data: Training data $X \in \mathbb{R}^{n \times d}$, labels $y \in \mathbb{R}^n$, number of boosting rounds T, learning rate η, subsample ratio s, maximum tree depth d_{max}
Result: Ensemble of trees $\mathcal{F}$

Initialize predictions $\hat{y}_i \leftarrow 0 \ \ \forall i \in \{1, \ldots, n\}$;
Initialize ensemble $\mathcal{F} \leftarrow \emptyset$;
for $t \leftarrow 1$ **to** T **do**
  for $i \leftarrow 1$ **to** n **do**
    $g_i \leftarrow \frac{\partial \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i}$;
    $h_i \leftarrow \frac{\partial^2 \ell(y_i, \hat{y}_i)}{\partial \hat{y}_i^2}$;
  end
  $T_t \leftarrow \texttt{BuildTree}(X, g, h, d_{max})$;
  Add tree T_t to ensemble $\mathcal{F}$;
  for $i \leftarrow 1$ **to** n **do**
    $\hat{y}_i \leftarrow \hat{y}_i + \eta \cdot T_t(X_i)$;
  end
end
return $\mathcal{F}$;

III. Architecture of FaGBM

A. Decision Tree and Random Forest Architecture

FaGBM trains trees layer-wise and consists of two stages: histogram counting and selecting the best split. FaGBM stores the histograms and tree nodes in on-chip memory. The tree nodes are compactly represented using 27–29 bits, storing the current best split (feature, threshold, and gain) along with child class labels. Leaf nodes are not explicitly stored, as each internal node contains the child class labels.

After initialization, the first stage of FaGBM reads in parallel all features and labels for multiple data instances from off-chip DRAM. ❶ This data is read using four input ports, each having a 512-bit bus width. ❷ Then, for each feature, it finds corresponding leaves by performing inference on the read data samples. For each leaf, we have two histograms with 2 dimensions (features, bins) to count instances for the appropriate labels (e.g. for binary classification, Labels 0 or 1).

[1]Tanaka *et al*. [23] published as a preprint in arXiv.

To reduce the number of LUTs, FaGBM time-multiplexes the features. In each histogram-counting iteration, histograms are computed for $NCOLS$ features (we define this parameter to control the amount of time-multiplexing). After finding leaves, ❸ histograms are updated based on the labels. The histogram counting stage is fully pipelined, allowing FaGBM to process one sample per cycle. As the bin value of each feature is updated in parallel, the input port count of the histogram memory must be greater than or equal to the feature count.

In the second phase, for each leaf, FaGBM finds the best feature and bin threshold value to split the leaf. The computation is executed for each feature in a pipelined fashion. ❹ First, multiple histograms mapped during the histogram-counting phase are reduced to a single histogram. ❺ Then, for each feature, cumulative summations of label 0 and label 1 counts are obtained. For this calculation, due to the bin-level parallelism, we need to separate the output ports for each bin. For the first stage, it was already required to have separate ports for each feature. With the addition of bin-level parallelism, BRAMs and URAMs cannot accommodate histograms because they allow only one word of data access per clock cycle. Thus, histograms are stored in LUTs.

We define the unique bin count of $NBINS$, thus each feature has $NBINS - 1$ threshold choices. ❻ The information gain for each threshold is calculated in parallel using the left and right counters for each label, obtained as their summation. For splits that result in zero instances on either the left or right leaf, the gain is set to the minimum value. The gains for each threshold are stored in registers. ❼ Afterward, the best threshold for the feature is found using a comparison tree. Using this method, the best gain value for each feature is determined at each clock cycle, and the best feature is updated accordingly. ❽ After finding the best split among $NCOLS$ features, the gain value of that split is compared with the current gain value of the node, and the split is replaced if it is greater. Finally, every node is written back to DRAM.

Bit-wise Approximate Logarithm — Equation 1 is defined in terms of logarithms of rational numbers (floating-point/fixed-point operations), but these can be substituted with the difference of two integer logarithms using a mathematical identity (log(a/b) = log(a) - log(b)). FaGBM runs in parallel $3 + 6 \times NBINS$ logarithms when the parent entropy is calculated. Even though fixed-point arithmetic provides some efficiency for logarithms, in some use cases, more aggressive approximations can be used to optimize area.

Bit-wise logarithms work in three steps [25]; see Figure 3. The first step calculates the largest power of two that the input includes, which corresponds to the bit index of the most significant bit of the input number. Then, this number is written to the integer part of the output. Finally, the next bits of the input are appended to the output after the decimal point. This calculation can be performed solely using LUTs, with no DSP core utilized, unlike the fixed-point/floating-point logarithms.

Random Forest — The hardware for random forest trains multiple trees using mostly the same design and the same

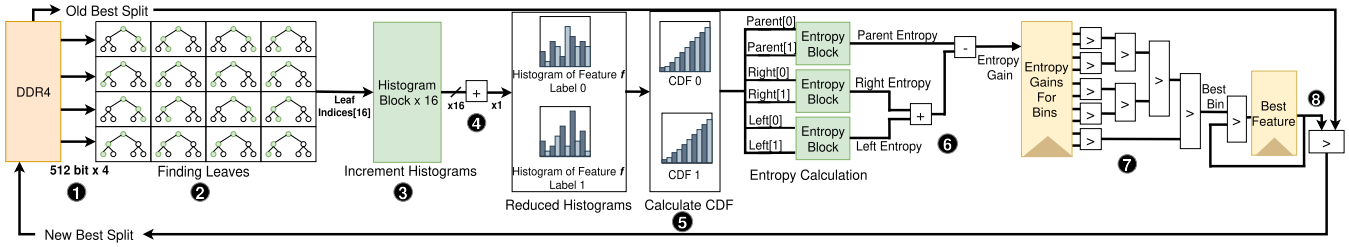


Fig. 2: System Diagram of FaGBM for Decision Tree with Binary Classification.

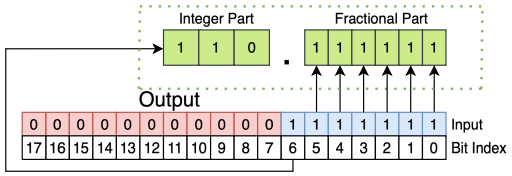


Fig. 3: Example of Bit-wise Approximate Logarithm.

logarithm approximation as the DT. However, due to bagging, FaGBM has an additional input port to read the feature indices for each tree. The randomly generated subset of features considered by the algorithm, $NCOLS$ indices, is passed as a parameter to FaGBM.

B. Gradient Boosted Decision Tree Architecture

Our proposed GBDT hardware shares some common sub-blocks with our DT hardware. In this section, we explain unique properties. First of all, each GBDT node is represented using 45–46 bits, storing the split information and a fixed-point node value. Since we store 40 trees with a depth of 6, each tree has 127 nodes. In total, 28KB of memory is required to store the trees. FaGBM needs instant access to only the tree being trained and the previous tree; thus, only those two trees are stored in LUTRAMs, and other trees are stored in DRAM. For histograms, we have $NCOLS \times NBINS$ accumulators for each of the leaves, and the accumulators have the same data type as node values. Histograms consume at most 1.75 MB of memory; they are stored in BRAMs. In addition, we need to store predictions for each data instance to use later for gradient and Hessian calculations. Each of these prediction values is represented as an 18-bit fixed-point value. For small datasets, we store them in BRAMs as well. However, for large datasets with more than 100K instances, it results in above 50% BRAM utilization, which causes timing and routing problems.

In the proposed system, Ⓐ we read features and labels from DRAM via an input port with a 256-1024-bit bus width, depending on the feature count and amount of parallelism. Then we set the prediction value to 0 during the first tree's training. For the next trees, the prediction value is read from BRAM if the dataset is small; otherwise, it is read from DRAM. Ⓑ Afterward, inference is performed on the previous tree, and the inferred leaf value is accumulated in the prediction. The updated prediction value is written back to BRAM for small datasets. For large datasets, we store each updated prediction in the register. When 16 predictions are updated, we write them back to BRAM as a 512-bit-wide word. Ⓒ Later, we compute gradient and Hessian values using the prediction values and accumulate gradient/Hessian histograms for each feature in parallel. For small datasets, hardware proceeds with the split computation. Ⓓ For large datasets, we offload BRAM contents to DRAM using wide words.

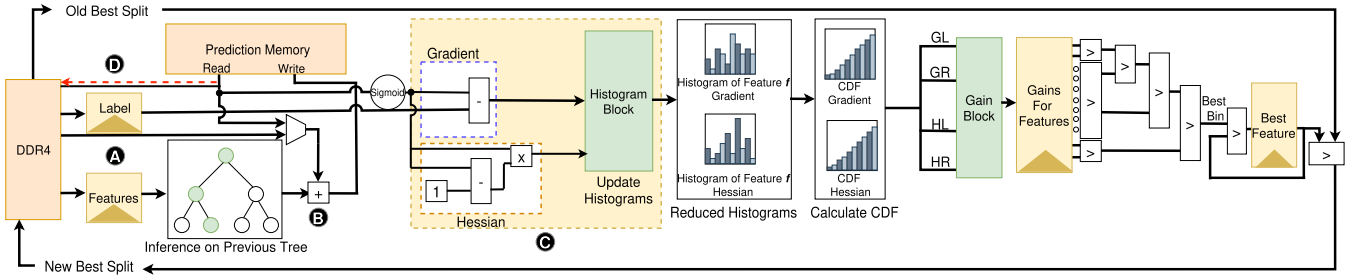
In the split-computing part, unlike our DT hardware, histograms have 256 bins, which makes unrolling the bin dimension impractical due to prohibitive LUTRAM area and routing overhead. Instead, we compute each feature's gain in parallel while pipelining the traversal of bins. When the feature count is large, even feature-level parallelism can lead to resource overutilization; therefore, we apply the same time-multiplexing strategy used in the DT hardware to maintain scalability. After finding the best split, the current tree is updated. When the final level of the current tree is constructed, previous tree content is written back to DRAM, and current tree content is written to the previous tree content.

1) Node Reuse - Harnessing Structural Similarity: Exploiting the structural similarity of trees in GBDT, FaGBM proposes a novel method to reuse tree nodes. In this method, all nodes in a layer are copied from the previous tree at a period. We defined a reuse factor to adjust the node reuse period. As depicted in Figure 5, when the reuse factor is two, the same node values are used for two consecutive trees. Our previous experiments showed that node feature diversity increases as the tree level decreases. Therefore, the reuse factor needs to decrease. In our method, we chose a heuristic that assigns nodes a reuse factor equal to half the reuse factor of their parents. Consequently, children's node features change more frequently than the parents'. From an algorithmic perspective, FaGBM calculates,

$$tree\ index * \mathrm{mod}\left(\frac{reuse\ factor}{2^{depth}}\right) \tag{3}$$

and if the result is zero, proceeds with histogram computing; otherwise, it copies all the nodes in the same level of the previous tree to the current tree.

2) Approximate Division: GBDT training involves three division operations to compute the gain; see Equation 2. However, the third division is the same across all splits; thus, it is omitted. FaGBM implements $2 \times NCOLS$ parallel division operations. This part causes high LUT utilization



The left part of the histogram block performs histogram counting. For large datasets, the prediction memory is offloaded to DDR4. The right-hand side of the histogram block implements split finding, with split calculations parallelized across features.

Fig. 4: System Diagram of FaGBM for Gradient Boosted Decision Trees.

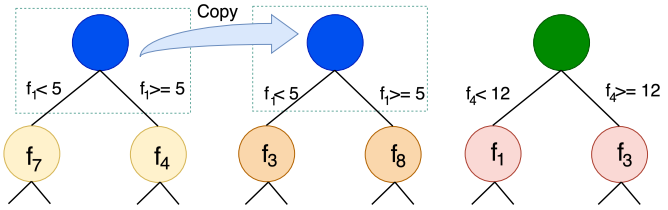


Fig. 5: Illustration of Node Reuse with Factor of 2.

when the fixed-point division is used. A more efficient option is to first take the reciprocal of the denominator using the *hls::recip* function, which uses the Newton-Raphson method, then multiply the result by the numerator. Yet, this method produces approximate results. The absolute error of this reciprocal approximation decreases when the number increases [29]. Building on this, we propose using approximate division for the larger denominator and fixed-point division for the other one. These denominators are the cumulative sums of Hessian values for the left and right children (HL, HR). Thus, they are complementary, and one increases when the other decreases.

IV. EXPERIMENTS

A. Experiment Setup

The experiments use 3 publicly available datasets selected as they appear repeatedly in the related FPGA literature. These are the HIGGS [30] – Cardinality 11M and dimensionality 28 –, MAGIC Gamma Telescope (MAGIC) [31] – Cardinality 19,020 and dimensionality 10 –, and Bank Marketing (Bank) [32] datasets – Cardinality 45,211 and dimensionality 51 – . The DT and RF experiments use a 163K subset of the HIGGS dataset, due to limitations of the Xilinx Data Analytics Library. Before execution, all datasets are summarised into histogram bins to match the hardware training pipeline. The DT and RF experiments use 10 bins per feature, whereas the GBDT experiments use 256 bins per feature. Except for HIGGS, where we follow the literature, datasets are split into 80/20 train–test. For GPUs, we use an NVIDIA RTX 2080 as a low-cost option for edge systems, as well as an NVIDIA Jetson Thor (GPU arch. Blackwell and 14-core Arm Neoverse V3AE). Jetson Thor is the latest edge platform from NVIDIA designed for edge workloads (e.g. robotics, autonomous systems). The GPU baselines use CUDA 12.4 with XGBoost 3.1.1, CatBoost 1.2.10, and LightGBM 4.6.0 on the RTX 2080, and CUDA 13.0 with XGBoost 3.2.1, CatBoost 1.2.10, and LightGBM 4.6.0 on the Jetson Thor. XGBoost 3.2.1 on the Jetson Thor was compiled from source (commit acb7b7f) because the 3.2.0 release did not support SM 110 GPUs.

FaGBM is synthesized with Xilinx Vitis HLS 2022.2 and implemented on an AMD Kintex™ Ultrascale+ KU15P FPGA and VCK5000 FPGA. During experiments, FPGA computation is initiated by an executable running on the host processor; for KU15P an Intel i9-7900X and AMD EPYC 3451 processor for VCK5000. For each experiment, datasets are first transferred to the FPGA's off-chip DDR4 memory via PCIe. Each experiment is repeated 5 times before measurements are recorded, and the runtime results are the average across 5 runs. Hardware resource utilizations are recorded after placement and routing. The reported clock frequencies are the kernel clock frequencies configured in the xclbin, at which all runtime measurements were taken.

B. Decision Tree Experiments

The DT experiments use three different hardware designs for each dataset. The first design uses fixed-point logarithms in entropy calculation. The second design uses the proposed bit-wise logarithm approximation. The third is the DT implementation in the Xilinx Data Analytics Library, which enables a direct comparison with an industrial-quality implementation. The LUT utilization increases linearly with the parameter $NCOLS$. The experiments use the maximum synthesizable $NCOLS$ values for each dataset: 5, 5, and 9 for the HIGGS, Magic, and Bank datasets, respectively.

Table I shows the utilization, accuracy, and frequency values. These results demonstrate that the bit-wise logarithm approximation incorporated into FaGBM (label Log approx) reduces DSP by up to 87% compared to the fixed-point implementation (Log fixed) without causing a significant drop in accuracy. As mentioned in Section II, the Xilinx implementation heavily utilizes URAMs. Due to space limitations, we were unable to report the URAM results. However, we have up to 89% lower utilization than the Xilinx baseline.

On the other hand, Xilinx's design uses fewer DSPs because it has limited parallelism (2-way) and uses the Gini Index as the split criterion. The Gini Index does not involve logarithms. FaGBM achieves the best runtime on the HIGGS and Magic datasets and the best accuracy on the Bank dataset. However, for the HIGGS and BANK datasets, there is a slight drop in accuracy due to the approximate logarithm.

TABLE I: FaGBM vs. Xilinx Decision Tree

	Accuracy	Time (ms)	LUT	BRAM	DSP	Freq (MHz)
Higgs KU15P						
Log approx	64.14%	3.91	287636	310	47	154.1
Log fixed	65.77%	3.95	292098	338.5	370	154.4
Xilinx	65.81%	16.23	215282	272	40	205
Higgs VCK5000						
Log approx	64.14%	2.27	176501	70	38	187.5
Log fixed	65.77%	2.36	198095	98.5	190	189.5
Xilinx	46.74%	32.6	119298	55.5	20	297.5
Magic KU15P						
Log approx	81.97%	0.25	259052	293.5	47	175.7
Log fixed	84.04%	0.26	265285	322	313	177.9
Xilinx	69.4%	1.49	211896	271	40	187
Magic VCK5000						
Log approx	81.97%	0.28	149013	52	38	216.1
Log fixed	84.04%	0.31	174520	80.5	133	221.7
Xilinx	69.4%	2.22	115922	54.5	20	284
Bank KU15P						
Log approx	89.44%	1.82	207838	290	47	190.9
Log fixed	89.49%	1.88	214688	318.5	370	188.6
Xilinx	89.28%	7.86	204615	271	43	207.3
Bank VCK5000						
Log approx	89.44%	1.13	94139	58	38	186.1
Log fixed	89.49%	1.35	124072	86.5	190	187.4
Xilinx	89.28%	17.08	110014	55.5	23	276.6

For completeness, on 16-cores, Higgs takes 259.34 ms with an accuracy 66.52%, Magic 11.93 ms with 83.62%, and Bank 38.6 ms with 89.82%.

C. Random Forest Experiments

The RF experiments use 100 trees to investigate the accuracy difference between approximate and fixed-point implementations of the logarithm. Conventionally, the $\sqrt{\#features}$ rule is used to select the number of features to train each node. After experimentation, we found that the best feature count for bagging is 5. As with the decision tree, we created 2 hardware implementations: the first with approximate and the second with fixed-point logarithms. As shown in Table II, using a majority vote across 100 trees reduces the accuracy gap between the approximate (indexed by 1) and fixed-point (indexed by 2) implementations for the Higgs and Magic datasets. For Bank, due to class imbalance and a large number of one-hot encoded features, bagging decreased accuracy.

D. Gradient Boosted Decision Tree Experiments

1) Experiments on Node Reuse: The experiments analyze the Reuse Factors between 1 and 50, as explained in Section III, where Reuse Factor = 1 denotes no Reuse; see Figure 6. For an accuracy loss of less than $<1\%$, the training time reduces up to 70%. For BANK and MAGIC, node reuse even increased accuracy.

TABLE II: Random Forest – Effect of Logarithm
FaGBM KU15P vs. multi-core with ensemble of 100 trees.

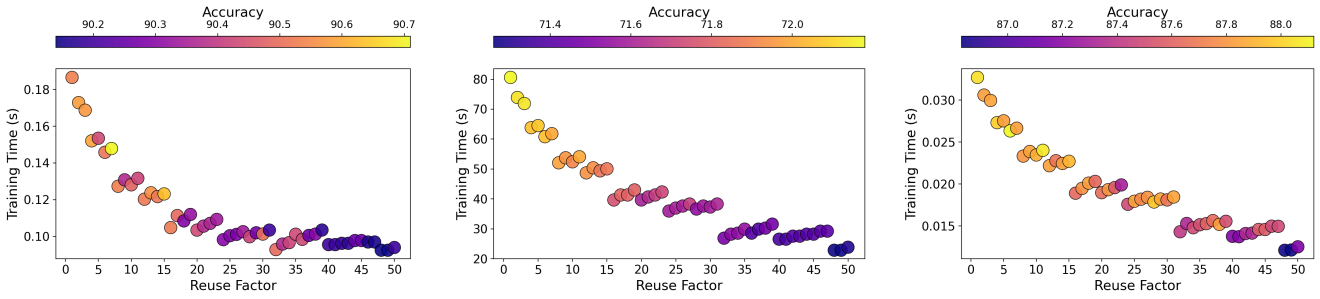
	Accuracy	Time
	Higgs	
FaGBM log approx	65.8%	19.19 ms
FaGBM log fixed	66.46%	21.07 ms
16-core	67.92%	365.28 ms
	Magic	
FaGBM log approx	83.78%	10.73 ms
FaGBM log fixed	84.8%	11.69 ms
16-core	84.31%	73.33 ms
	Bank	
FaGBM log approx	88.3%	16.66 ms
FaGBM log fixed	88.3%	17.28 ms
16-core	89.82%	87.39 ms

TABLE III: FaGBM – Effect of Division Approximation for GBDT on KU15P FPGA

			Higgs		
Dataset	**Accuracy**	**LUT**	**BRAM**	**DSP**	**Freq (MHz)**
Div fixed	72.18%	318054	755	251	178.2
Div adap	71.72%	276306	755	307	173.7
Div approx	53.3%	223022	755	363	162.3
Div hybrid	70.69%	273308	755	307	182.4
			Magic		
Div fixed	88.12%	202233	395.5	108	226
Div adap	88.33%	188520	395.5	128	229.2
Div approx	86.88%	170318	395.5	148	222.3
Div hybrid	87.91%	187093	395.5	128	221.2
			Bank		
Div fixed	90.71%	245701	546.5	164	206.2
Div adap	90.41%	221113	546.5	196	201.7
Div approx	89.95%	190203	546.5	230	221.6
Div hybrid	90.30%	218850	546.5	192	208.5

2) Impact of Approximate Division: FaGBM has 4 different division techniques implemented. The first three are: fixed point division (denoted by fixed), adaptive division as explained in Section III (denoted *adap*), and approximate division (denoted *approx*). The last one uses hybrid division, where we apply approximate division for HL and fixed division for HR (denoted by *hybrid*). Table III displays the results. Using adaptive division, FaGBM achieved up to 40K savings in LUTs with less than 0.5% drop in accuracy compared to fixed-point division. With approx, the accuracy degradation worsened and even caused numerical instability on the HIGGS dataset, leading to incorrect results. On the other hand, the hybrid technique provides a fair trade-off, yielding accuracy and area results that fall between those of adaptive and approximate techniques.

3) Performance Comparisons against XGBOOST, LightGBM, and CatBoost: The next set of experiments compares FaGBM with different industry-optimized GPU baselines in terms of accuracy and runtime. The models are trained on the entire HIGGS dataset to compare against GPU and multi-core baselines, and Table IV presents the results. For the VCK5000 platform, we additionally evaluate an 8-way parallel histogram computation configuration of FaGBM to leverage the increased available resources. XGBOOST-Exact denotes running



(a) Bank FaGBM (b) Higgs FaGBM (c) Magic FaGBM

Fig. 6: FaGBM - Analysis of accuracy vs. runtime for reuse factors in the range 1..50 on KU15P FPGA.

TABLE IV: Higgs — FaGBM vs multi-core and GPUs.

Method	Runtime	Accuracy	RMSE	Device
CatBoost	48.5 s	70.85%	0.4376	1-Core
CatBoost	7.35 s	70.87%	0.4378	16-Core
CatBoost	5.72 s	70.81%	0.4378	AGX Thor
CatBoost	3.05 s	70.81%	0.4378	RTX 2080
XGBoost-Exact*	928.96 s	72.23%	0.4283	1-Core
XGBoost-Exact*	81.68 s	72.23%	0.4283	16-Core
XGBoost	31.76 s	72.27%	0.4282	1-Core
XGBoost	6.43 s	72.27%	0.4282	16-Core
XGBoost	5.16 s	72.26%	0.4283	AGX Thor
XGBoost	2.4 s	72.26%	0.4283	RTX 2080
LightGBM	35.11 s	72.26%	0.4283	1-Core
LightGBM	4.73 s	72.26%	0.4283	16-Core
LightGBM	7.74 s	72.73%	0.4247	AGX Thor
LightGBM	5.68 s	72.73%	0.4247	RTX 2080
FaGBM				
1-way parallel	24.14 s	71.29%	0.4339	KU15P
1-way parallel	8.93 s	71.29%	0.4339	VCK5000
8-way parallel	2.34 s	71.29%	0.4339	VCK5000
1-way no Node Reuse	81.59 s	72.18%	0.4285	KU15P
1-way no Node Reuse	30.18 s	72.18%	0.4285	VCK5000
8-way no Node Reuse	7.9 s	72.18%	0.4285	VCK5000

* XGBoost-Exact is not supported for GPUs.

TABLE V: Power Consumption for Higgs.

Platform	Measurement Type	Power (W)
	XGBOOST	
RTX 2080	On-board sensor	100.6
AGX Thor	On-board sensor	16
CPU 16-Core (Histogram)	Wall Power	87.68
CPU 16-Core (Exact)	Wall Power	106.73
CPU 1-Core (Histogram)	Wall Power	29.76
	FaGBM	
Dynamic Power KU15P	Power-rail	1.33
Static Power KU15P	Power-rail	17.48
Dynamic Power VCK5000	Power-rail	4.5
Static Power VCK5000	Power-rail	39.6

XGBOOST without any binning, while simply XGBOOST denotes using the histogram approximation with 256 bins. Compared with baselines that compute split values exactly, FaGBM is up to 2.2x faster than the XGBoost GPU and up to 2.74x faster than the multi-core XGBoost implementations, with similar accuracy. For approximate, the XGBoost library on the RTX 2080 is the only one that comes close to FaGBM times on the VCK5000.

4) Power Measurement Methodology and Comparison: The power consumption of the FaGBM architecture is evaluated directly from the FPGA power rails. For static power, we measured power in the idle state. For dynamic power, we measured power during execution and subtracted the static power. For comparison, we also measured the dynamic power consumption of XGBoost on RTX 2080, AGX Thor GPUs using *nvidia-smi* [33] and reported the results in Table V. We also measured dynamic CPU power on an AMD Ryzen AI MAX+ 395 while running XGBoost with the *XGBOOST-Exact* and Histogram-based XGBOOST configurations. Overall, FaGBM achieves a dynamic power consumption of just 1.33–4.5 W, and total power of 18.81-44.1 W.

V. Conclusions

This work proposed FaGBM, the first scalable, FPGA accelerator for training decision trees, random forests, and GBDT. We have analyzed the structure of tree-based models under a hardware mindset. Upon this inspection, we identified a set of key inefficiencies: redundant re-computation of nodes, costly logarithm operations and fixed-point divisions, and targeted them with FPGA optimizations. For Decision Tree and Random Forest Training, FaGBM achieves up to 87% reduction in DSPs utilizing bitwise logarithms. In GBDT training, using Node Reuse reduced training time by up to 70%, and using Adaptive Division decreased LUT utilization by providing a clear trade-off between LUT and DSP. Across all models, these optimizations preserve accuracy within acceptable bounds. Compared to software baselines, FaGBM on VCK5000 achieves 2x speedup over NVIDIA Jetson Thor. After detailed power-consumption experiments, FaGBM also demonstrates the highest energy efficiency among existing hardware solutions for tree-based model training.

Acknowledgments

This work is partially funded by EPSRC EP/N035127/1 (LAMBDA project) and EP/T026995/1 (EnnCore project). Mikel Luján is supported by a Royal Society Wolfson Fellowship and an Arm/RAEng Research Chair Award.

REFERENCES

[1] J. R. Quinlan, "Induction of decision trees," *Machine learning*, vol. 1, no. 1, pp. 81–106, 1986.

[2] L. Breiman, J. Friedman, R. A. Olshen, and C. J. Stone, *Classification and regression trees*. Chapman and Hall/CRC, 2017.

[3] J. Chen, H. M. Le, P. Carr, Y. Yue, and J. J. Little, "Learning online smooth predictors for realtime camera planning using recurrent decision trees," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4688–4696.

[4] S. Y. Kim and A. Upneja, "Predicting restaurant financial distress using decision tree and adaboosted decision tree models," *Economic Modelling*, vol. 36, pp. 354–362, 2014.

[5] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[6] J. H. Friedman, "Greedy function approximation: a gradient boosting machine," *Annals of statistics*, pp. 1189–1232, 2001.

[7] ——, "Stochastic gradient boosting," *Computational statistics & data analysis*, vol. 38, no. 4, pp. 367–378, 2002.

[8] T. Chen, "Xgboost: A scalable tree boosting system," *Cornell University*, 2016.

[9] L. Xu, J. Liu, and Y. Gu, "A recommendation system based on extreme gradient boosting classifier," in *2018 10th International Conference on Modelling, Identification and Control (ICMIC)*. IEEE, 2018, pp. 1–5.

[10] S. Li and X. Zhang, "Research on orthopedic auxiliary classification and prediction model based on xgboost algorithm," *Neural Computing and Applications*, vol. 32, no. 7, pp. 1971–1979, 2020.

[11] A. V. Dorogush, V. Ershov, and A. Gulin, "Catboost: gradient boosting with categorical features support," *arXiv preprint arXiv:1810.11363*, 2018.

[12] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "Lightgbm: A highly efficient gradient boosting decision tree," *Advances in neural information processing systems*, vol. 30, 2017.

[13] T. P. Dinh, C. Pham-Quoc, T. N. Thinh, B. K. Do Nguyen, and P. C. Kha, "A flexible and efficient fpga-based random forest architecture for iot applications," *Internet of Things*, vol. 22, p. 100813, 2023.

[14] P. Serhiayenka, S. T. Roche, B. T. Carlson, and T. M. Hong, "Nanosecond hardware regression trees in fpga at the lhc," *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment*, vol. 1072, p. 170209, 2025.

[15] A. Khataei and K. Bazargan, "Treelut: An efficient alternative to deep neural networks for inference acceleration using gradient boosted decision trees," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 14–24.

[16] M. Shah, R. Neff, H. Wu, M. Minutoli, A. Tumeo, and M. Becchi, "Accelerating random forest classification on gpu and fpga," in *Proceedings of the 51st International Conference on Parallel Processing*, 2022, pp. 1–11.

[17] M. Owaida and G. Alonso, "Application partitioning on fpga clusters: Inference over decision tree ensembles," in *2018 28th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2018, pp. 295–2955.

[18] A. Alcolea and J. Resano, "Fpga accelerator for gradient boosting decision trees," *Electronics*, vol. 10, no. 3, p. 314, 2021.

[19] M. Owaida, H. Zhang, C. Zhang, and G. Alonso, "Scalable inference of decision tree ensembles: Flexible design for cpu-fpga platforms," in *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2017, pp. 1–8.

[20] S. Summers, G. Di Guglielmo, J. Duarte, P. Harris, D. Hoang, S. Jindariani, E. Kreinar, V. Loncar, J. Ngadiuba, M. Pierini *et al.*, "Fast inference of boosted decision trees in fpgas for particle physics," *Journal of Instrumentation*, vol. 15, no. 05, p. P05026, 2020.

[21] Z. Wen, J. Shi, B. He, J. Chen, K. Ramamohanarao, and Q. Li, "Exploiting gpus for efficient gradient boosting decision tree training," *IEEE Transactions on Parallel and Distributed Systems*, vol. 30, no. 12, pp. 2706–2717, 2019.

[22] M. He, M. Thottethodi, and T. Vijaykumar, "Booster: An accelerator for gradient boosting decision trees training and inference," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2022, pp. 1051–1062.

[23] T. Tanaka, R. Kasahara, and D. Kobayashi, "Efficient logic architecture in training gradient boosting decision tree for high-performance and edge computing," *arXiv preprint arXiv:1812.08295*, 2018.

[24] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[25] J. N. Mitchell, "Computer multiplication and division using binary logarithms," *IRE Transactions on Electronic Computers*, no. 4, pp. 512–517, 2009.

[26] Xilinx, Inc., "Decision tree (training) — vitis data analytics library 2022.1," https://xilinx.github.io/Vitis_Libraries/data_analytics/2022.1/guide_L2/internals/DT.html, 2022, accessed: 2025-12-31.

[27] Z. Lin, S. Sinha, and W. Zhang, "Towards efficient and scalable acceleration of online decision tree learning on fpga," in *2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2019, pp. 172–180.

[28] Z. Wen, J. Shi, B. He, Q. Li, and J. Chen, "ThunderGBM: Fast GBDTs and random forests on GPUs," *Journal of Machine Learning Research*, vol. 21, 2020.

[29] R. L. Burden and J. D. Faires, *Numerical analysis*. Brooks Cole, 1997.

[30] D. Whiteson, "HIGGS," UCI Machine Learning Repository, 2014, DOI: https://doi.org/10.24432/C5V312.

[31] R. Bock, "MAGIC Gamma Telescope," UCI Machine Learning Repository, 2004, DOI: https://doi.org/10.24432/C52C8B.

[32] R. P. Moro, S. and P. Cortez, "Bank Marketing," UCI Machine Learning Repository, 2014, DOI: https://doi.org/10.24432/C5K306.

[33] NVIDIA Corporation, *NVIDIA System Management Interface (nvidia-smi)*, NVIDIA, 2025, accessed: 2026-01-17. [Online]. Available: https://docs.nvidia.com/deploy/nvidia-smi/index.html